QUIZ – SECTION BCS-4F

Name:_____ Roll No:_____

Part A: Multiple Choice Questions (10 × 1 = 10 Marks)

1.In a pipe, data flows:

A. In both directions automatically
B. In one direction (unidirectional)
C. Randomly
D. Only from parent to parent

2. Consider:

```
if (fork() == 0) {
    exit(10);
}
```

What value can parent retrieve?
A) 0 B) 10 C) PID D) 255

3. Which header file is required for `wait()` and `waitpid()`?
A) stdio.h B) unistd.h C) sys/wait.h D) stdlib.h

4. If a child is killed by a signal, `WIFEXITED(status)` returns:
A) 1 B) 0 C) PID D) Exit code

5. Which statement about exec() is TRUE?
A) It creates child process
B) It returns twice
C) It replaces process image
D) It duplicates process

6. What must be the last argument of execlp()?
A) 0 B) '\0' C) NULL D) -1

7. What is the purpose of exit status?
A) Data transfer of large arrays
B) Communicate small result to parent
C) Store PID
D) Replace process

8. After calling fork(), how many processes exist?
A) 0 B) 1 C) 2 D) 3

9. `wait(NULL);` means:
A) Do not wait B) Wait but ignore status C) Kill child D) Restart child

10. Which call allows controlled synchronization of specific child processes?
A) fork() B) exit() C) waitpid() D) execlp()

a·5

Part B: Short Questions (5 × 2 = 10 Marks)

1. How does the `wait()` function help in process synchronization?

When wait() is called, the immediate child of that parent (who calls wait()) will execute first. So, the process becomes synchronized as the parent executes after the child.

2. What is an unnamed pipe? Explain how data flows between parent and child processes using it.

Unnamed pipe or simply pipe are a way to communicate between child and parent through file descriptors. Every process has its own file descriptor table. Unnamed pipes have one way communication through read() and write() system calls.

In code:

```
int fd[2];
pipe(fd);   (// creates pipe in which fd[0] is read end and fd[1] is write end.
             (It has alot detail more).
```

3. What is the purpose of the macros:

- `WIFEXITED` and `WEXITSTATUS`

→ WEXITSTATUS() is for printing the (exit code) status value as it can't directly as status because it has other things also.

→ WIFEXITED() is for checking whether the child process has been exited or not.

4. Why can exit() not return large values like 1000?

exit() just return upto 255 values because of the datatype as it has int datatype (or something else maybe). Therefore, it can't return large values.

5. Consider the following code:

```
if (fork() == 0) {
    exit(25);
}
wait(&status);
printf("%d\n", WEXITSTATUS(status));
```

What will be printed?

25